

ROTEIRO DE APRENDIZAGEM**Projeto Jogo da Memória
com HTML, CSS e JavaScript**

Projeto introdutório para desenvolvimento web

Curso		Técnico em Desenvolvimento de Sistemas
Componente conteúdo	/	Programação Web — HTML, CSS e JavaScript
Turma		1ª série do Ensino Médio
Instituição		Etec de Olímpia

Objetivo

Construir um jogo da memória simples, utilizando HTML para a estrutura, CSS para a aparência e JavaScript para a interação e a lógica do jogo.

Ao final, o jogo deverá:

1. Exibir várias cartas viradas para baixo.
2. Permitir virar uma carta ao clicar.
3. Permitir escolher uma segunda carta.
4. Comparar as duas cartas.
5. Manter abertas as cartas quando forem iguais.
6. Esconder novamente as cartas quando forem diferentes.
7. Continuar até que todos os pares sejam encontrados.

Como usar este roteiro: avance uma etapa por vez. Teste o resultado no navegador antes de seguir. Quando algo não funcionar, chame um dos professores para auxiliar.

Etapa 1 — Entendendo o jogo

Antes de programar, pense em como funciona um jogo da memória.



Cada figura aparece duas vezes. O jogador vira duas cartas por vez.

Se forem iguais: as duas cartas permanecem abertas. Exemplo: 🐶 = 🐶

Se forem diferentes: depois de alguns instantes, as duas cartas voltam a ficar escondidas. Exemplo: 🐶 ≠ 🐱

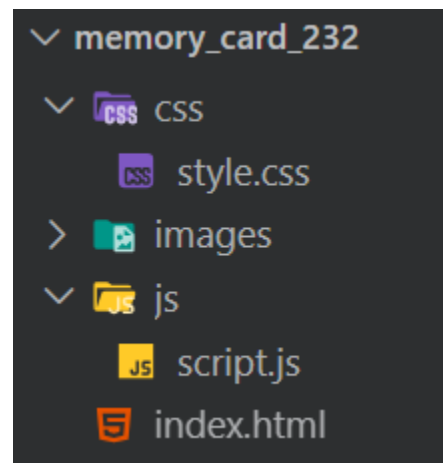
Etapa 2 — Criando os arquivos

Crie uma pasta para o projeto e, dentro dela, três pastas contendo os seguintes arquivos base:

- index.html: estrutura da página e das cartas.
- style.css: aparência e organização visual.
- script.js: funcionamento e regras do jogo.

Etapa 3 — Criando a página HTML

Crie a estrutura básica da página. Ela deve possuir um título, uma área para as cartas e a ligação com o arquivo JavaScript. Não se esqueça de ligar também o CSS no <head>.



```
<body>
  <h1>Jogo da memória</h1>
  <section class="tabuleiro">
    <section class="cartao">
      
      
    </section>
  </section>
</body>
```

A `<section class="tabuleiro">` será o espaço onde colocaremos todos os cartões.

Seu desafio

Crie dentro do tabuleiro 6 cartas. Cada carta pode começar assim:

```
<section class="cartao">
  ?
</section>
```

Confira antes de avançar: o título apareceu? As 6 cartas aparecem no navegador?

Etapa 4 — Organizando as cartas com CSS

Agora faça os elementos parecerem cartas. Comece pela classe `.carta` e experimente largura, altura, borda, cor de fundo, tamanho da fonte e alinhamento.

```
.carta {
  /* escreva seus estilos aqui */
}
```

Objetivo visual:

```
+-----+ +-----+ +-----+
| ? || ? || ? |
+-----+ +-----+ +-----+

+-----+ +-----+ +-----+
| ? || ? || ? |
+-----+ +-----+ +-----+
```

Etapa 5 — Identificando cada carta

Precisamos informar ao JavaScript qual símbolo está escondido em cada carta. Podemos usar um atributo `data-*` no HTML.

```
<section class="cartao" data-framework="🐶">?</section>
<section class="cartao" data-framework="🐱">?</section>
```

Monte 12 cartas utilizando 6 pares: 🐶 🐶, 🐱 🐱 e 🐸 🐸. Misture as posições, por exemplo:

🐶 🐱 🐸 🐱 🐶 🐸

Pergunta: por que o símbolo fica em `data-framework`, enquanto a carta mostra “?”? Porque a informação precisa existir no elemento, mas deve permanecer escondida até o clique.

Etapa 6 — Capturando as cartas com JavaScript

No arquivo `script.js`, precisamos localizar todas as cartas da página:

```
const cartoes = document.querySelectorAll(".cartao");
console.log(cartoes);
```

Abra as Ferramentas do Desenvolvedor do navegador (geralmente F12) e procure a aba Console.

Confira: o JavaScript encontrou todas as cartas?

Etapa 7 — Detectando o clique

Precisamos executar uma ação sempre que uma carta for clicada. Para cada carta, adicione um evento de clique:

```
cartas.forEach(function(carta) {

  carta.addEventListener("click", function() {
    console.log("Carta clicada");
  });

});
```

Teste clicando em várias cartas.

Desafio: em vez de mostrar “Carta clicada”, mostre no Console o símbolo da carta clicada. Uma pista é usar `carta.dataset.simbolo`.

Etapa 8 — Virando uma carta

Quando o jogador clicar, queremos trocar o “?” pelo símbolo escondido.

Uma propriedade útil para alterar o conteúdo textual de um elemento é `textContent`.

```
carta.textContent = ...;
```

O valor de que você precisa está em `carta.dataset.simbolo`.

Objetivo da etapa: ao clicar nas cartas, cada uma deve revelar seu símbolo. Ainda não precisamos esconder as cartas novamente.

Etapa 9 — Guardando a primeira carta

Agora precisamos lembrar qual foi a primeira carta escolhida pelo jogador. Crie uma variável antes dos eventos de clique:

```
let primeiraCarta = null;
```

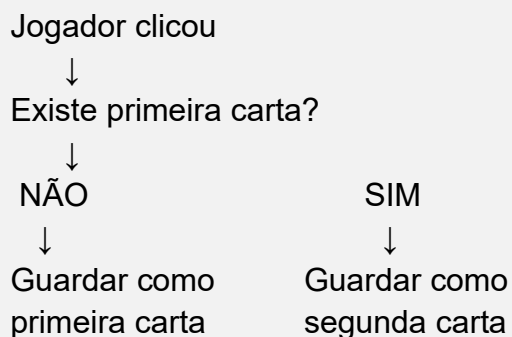
Quando uma carta for clicada, verifique se ainda não existe uma primeira carta escolhida. Nesse caso, guarde a carta clicada em `primeiraCarta`.

Teste: use `console.log(primeiraCarta)` e observe o elemento armazenado.

Etapa 10 — Guardando a segunda carta

```
let segundaCarta = null;
```

A lógica começa a ficar assim:



Quando existirem duas cartas escolhidas, teremos informações suficientes para compará-las.

Etapa 11 — Comparando as cartas

Precisamos verificar se o símbolo da primeira carta é igual ao símbolo da segunda carta.

```
primeiraCarta.dataset.simbolo  
segundaCarta.dataset.simbolo
```

Utilize um if. Pense nesta estrutura:

```
if (alguma_coisa === alguma_coisa) {  
    console.log("Acertou!");  
} else {  
    console.log("Errou!");  
}
```

Teste: clique em duas cartas iguais e depois em duas diferentes. O Console precisa informar corretamente “Acertou!” ou “Errou!”.

Etapa 12 — Mantendo os pares corretos

Quando as cartas forem iguais, elas podem permanecer mostrando os símbolos. Depois disso, precisamos preparar o jogo para uma nova tentativa:

```
primeiraCarta = null;  
segundaCarta = null;
```

Etapa 13 — Escondendo cartas diferentes

Quando as cartas forem diferentes, precisamos esperar um pouco antes de escondê-las novamente. JavaScript possui a função `setTimeout()`.

```
setTimeout(function() {  
    console.log("Passou um tempo!");  
}, 1000);
```

O valor 1000 representa aproximadamente 1 segundo. Dentro do `setTimeout`, faça a primeira e a segunda cartas voltarem a mostrar “?” e, em seguida, limpe as duas variáveis.

```
primeiraCarta = null;  
segundaCarta = null;
```

Etapa 14 — Impedindo erros

Problema 1: clicar duas vezes na mesma carta

O jogador não deve poder formar um par clicando duas vezes na mesma carta. Crie uma condição que interrompa o clique se a carta clicada for exatamente a mesma que primeiraCarta.

Problema 2: clicar numa terceira carta durante a comparação

Podemos criar uma variável para bloquear temporariamente novos cliques:

```
let bloqueado = false;
```

Quando duas cartas diferentes forem escolhidas, use bloqueado = true. Depois que elas forem escondidas novamente, use bloqueado = false.

No início do evento de clique:

```
if (bloqueado) {
  return;
}
```

Etapa 15 — Melhorando o visual

Quando a lógica estiver funcionando, melhore a aparência. Experimente:

```
border-radius
box-shadow
background-color
font-size
gap
transition
```

Você também pode criar uma classe para representar visualmente uma carta aberta:

```
.carta.virada {
  /* estilo da carta aberta */
}
```

Etapa 16 — Organizando o raciocínio

Observe o algoritmo completo antes de revisar seu código:

INÍCIO

Jogador clica em uma carta



Mostrar símbolo



É a primeira carta?

SIM → guardar primeira carta

NÃO → guardar segunda carta



Comparar as duas



São iguais?

SIM

→ manter abertas

→ limpar primeiraCarta

→ limpar segundaCarta

NÃO

→ esperar

→ esconder as duas

→ limpar primeiraCarta

→ limpar segundaCarta

Ideia central: esse algoritmo é o coração do jogo. Se ele estiver claro, o código fica muito mais fácil de compreender e corrigir.

Etapa 17 — Teste do jogo

Antes de considerar o projeto concluído, faça estes testes:

- ☐ Todas as cartas aparecem.
- ☐ Todas possuem um par.
- ☐ É possível virar uma carta.
- ☐ É possível escolher uma segunda carta.
- ☐ Duas cartas iguais permanecem abertas.
- ☐ Duas cartas diferentes são escondidas novamente.
- ☐ Não é possível utilizar a mesma carta como um par.

- ☐ O jogo continua funcionando depois de vários cliques.
- ☐ Todos os pares podem ser encontrados.

Desafios de evolução

Desafio 1 — Contador de jogadas

Adicione na página:

```
<p>Jogadas: <span id="jogadas">0</span></p>
```

Crie uma variável e aumente seu valor sempre que duas cartas forem escolhidas:

```
let jogadas = 0;  
  
// quando completar uma tentativa:  
jogadas++;
```

Depois atualize o valor exibido na página.

Desafio 2 — Mensagem de vitória

```
let paresEncontrados = 0;
```

Sempre que o jogador acertar um par, aumente essa variável. Como o jogo inicial tem três pares, verifique quando paresEncontrados chegar a 3 e mostre uma mensagem de vitória.

 Parabéns! Você encontrou todos os pares!

Desafio 3 — Embaralhar as cartas

Até agora, as cartas provavelmente aparecem sempre na mesma posição. Pesquise `Math.random()` e investigue uma maneira de mudar a ordem das cartas a cada partida.

Desafio 4 — Criar as cartas pelo JavaScript

Em vez de escrever todas as cartas no HTML, crie um vetor:

```
const simbolos = [
  "🐶",
  "🐱",
  "🐸",
  "🐶",
  "🐱",
  "🐸"
];
```

Depois, pesquise `document.createElement()` e `appendChild()` para gerar as cartas automaticamente.

Desafio 5 — Aumentar o jogo

Quando a versão com 6 cartas funcionar, experimente:

- 8 cartas → 4 pares
- 12 cartas → 6 pares
- 16 cartas → 8 pares

Missão final

Seu jogo deve possuir, no mínimo:

HTML	CSS	JavaScript
Título	Organização das cartas	Evento de clique
Tabuleiro	Tamanho das cartas	Primeira carta
Cartas	Cores e espaçamento	Segunda carta
	Efeito ao passar o mouse	Comparação
		Tratamento de acerto e erro

Perguntas para reflexão

1. Para que utilizamos `querySelectorAll()`?

2. Para que serve `addEventListener()`?

3. O que significa o evento "click"?

4. Por que precisamos da variável primeiraCarta?

5. Por que precisamos da variável segundaCarta?

6. Para que utilizamos o if?

7. Qual é a função do setTimeout()?

8. O que significa carta.dataset.simbolo?

9. Por que precisamos colocar novamente null nas variáveis das cartas?

10. Qual foi a parte mais difícil de desenvolver?

Sugestão de sequência de aulas

Aula	Foco	Resultado esperado
1	HTML + CSS	Construir o tabuleiro e estilizar as cartas.
2	Eventos JavaScript	Detectar cliques e revelar os símbolos.
3	Lógica do jogo	Guardar duas cartas, comparar, acertar e esconder.
4	Melhorias	Contador, vitória, embaralhamento e visual.